Studio Nx

JS 282 — 클립 색인과 타임라인 클립 그리기

2026-10-19

JavaScript 282 — 클립 색인과 타임라인 클립 그리기 (sequencer-devices.js)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-10a 클립·연결·패턴 API: `/full` 클립 색인, 편곡 클립 요소 만들기, MIDI 미리보기, 펼침 칸 다시 채우기 (PRD 02 FR-2196 클립·패턴, FR-2197 연결)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 273 `sequencer-devices.js` 전체 채우기, 다음 283 Connect 칸

1. 하는 일

`GET /projects/{id}/full` 로 받은 `clips` 와 `patterns` 를 모듈 안에 담아 두고, 트랙을 그릴 때 그 트랙의 클립만 골라 `createClip` 으로 편곡 화면에 붙입니다. MIDI 클립은 노트를 `canvas.sq-clip__preview` 에 작은 막대로 그립니다. 그리고 `/full` 을 다시 받은 뒤 열려 있는 펼침 칸만 다시 채웁니다.

```
GET /projects/{id}/full
  ├─ patterns → indexPatterns → patternsById[pattern_id]
  ├─ clips    → indexClips    → clips 배열
  └─ tracks   → renderTracks
        ↓
  clipsFor(track) — 한 트랙의 클립만 (start_tick 순)
        ↓
  clipElement(track, c) — createClip({ …, clipId, patternId, sampleId })
        ↓ button.sq-clip (.sq-clip--midi | .sq-clip--audio)
  drawClipPreviews() → drawMidiPreview(clip) — canvas 에 노트 막대

applyFullProject / refreshConnections
        ↓
  refreshOpenCells() — .is-open 인 트랙 줄만 ensureLiveCells
```

2. 언제 쓰나

쓰는 곳	
`applyFullProject(data)`	`/full` 의 patterns 와 clips 를 각각 `indexPatterns`, `indexClips` 로 담습니다
`renderTrack` 이 펼침 칸을 만들 때	`clipsFor(track)` 로 그 트랙의 클립을 `clipElement` 로 그립니다
레이아웃이 그려진 뒤	`drawClipPreviews()` 가 모든 MIDI 클립 미리보기를 다시 그립니다
`applyFullProject` · `initDevices` · `refreshConnections`	`refreshOpenCells()` 로 펼쳐 둔 칸만 다시 채웁니다
`sequencer.js` 저장 경로	클립의 `dataset.clipId`, `dataset.patternId`, `dataset.sampleId` 를 읽어 PATCH · DELETE 에 씁니다

3. 코드

```
/** 패턴(노트 포함)을 번호로 담기 (S1-10a) — MIDI 클립 이름 · 미리보기에 씀 */
function indexPatterns(list) {
  (list || []).forEach(function (pattern) { patternsById[pattern.pattern_id] = pattern; });
}

/** 프로젝트의 모든 클립을 담기 (S1-10a FR-2196) */
function indexClips(list) {
  clips = (list || []).slice();
}

/** 한 트랙의 클립 (시작 tick 순) */
function clipsFor(track) {
  return clips.filter(function (c) { return String(c.track_id) === String(track.track_id); })
    .sort(function (a, b) { return a.start_tick - b.start_tick; });
}

/** 서버 클립 한 건 → 편곡 클립 요소 (번호를 달아 두어 저장·삭제가 이어짐) */
function clipElement(track, c) {
  const kind = c.clip_type === 'AUDIO' ? 'audio' : 'midi';
  const el = createClip({
    kind: kind,
    name: clipName(c),
    start: ticksToBars(c.start_tick),
    length: ticksToBars(c.length_tick),
    clipId: c.clip_id,
    patternId: c.pattern_id,
    sampleId: c.sample_id,
  });
  if (c.offset_tick) el.style.setProperty('--o', ticksToBars(c.offset_tick));
  if (kind === 'midi' && c.pattern_id && patternsById[c.pattern_id]) {
    const p = patternsById[c.pattern_id];
    el.__notes = p.notes || [];
    if (p.length_tick) el.__patternBars = Math.max(1, ticksToBars(p.length_tick));
  }
  return el;
}

/** 클립 이름 — 저장된 이름이 없으면 패턴 이름 또는 'Sample #N' */
function clipName(c) {
  if (c.name) return c.name;
  if (c.pattern_id) {
    const p = patternsById[c.pattern_id];
    return p ? p.name : 'Pattern #' + c.pattern_id;
  }
  return 'Sample #' + c.sample_id;
}

/** MIDI 클립 미리보기 다시 그리기 (FR-2196) — __notes 의 노트를 음높이·길이대로 작은 막대로 */
function drawMidiPreview(clip) {
  const notes = clip.__notes;
  const canvas = clip.querySelector('.sq-clip__preview');
  if (!canvas || !notes) return;
  const dpr = window.devicePixelRatio || 1;
  const w = canvas.clientWidth;
```

```
  const h = canvas.clientHeight;
  const pw = Math.max(1, Math.round(w * dpr));
  const ph = Math.max(1, Math.round(h * dpr));
  if (canvas.width !== pw || canvas.height !== ph) { canvas.width = pw; canvas.height = ph; }
  const ctx = canvas.getContext('2d');
  ctx.setTransform(1, 0, 0, 1, 0, 0);
  ctx.clearRect(0, 0, pw, ph);
  if (w < 2 || h < 2 || !notes.length) return;
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
  const ticks = (parseFloat(clip.style.getPropertyValue('--l')) || 1) * TICKS_PER_BAR;
  let lo = Infinity;
  let hi = -Infinity;
  for (let i = 0; i < notes.length; i += 1) { if (notes[i].pitch < lo) lo = notes[i].pitch; if (notes
[i].pitch > hi) hi = notes[i].pitch; }
  if (hi === lo) { lo -= 1; hi += 1; }
  const pad = 0.12;
  const barH = Math.max(1, Math.round(h * 0.14));
  ctx.fillStyle = getComputedStyle(clip).getPropertyValue('--tc').trim() || 'currentColor';
  for (let i = 0; i < notes.length; i += 1) {
    const n = notes[i];
    const x = (n.start_tick / ticks) * w;
    const bw = Math.max(1, (n.length_tick / ticks) * w);
    const y = (1 - (n.pitch - lo) / (hi - lo)) * (1 - pad * 2) + pad;
    ctx.fillRect(x, y * h - barH / 2, Math.min(bw, w - x), barH);
  }
}

/** 레이아웃이 그린 뒤 모든 MIDI 클립 미리보기를 다시 그리기 (캔버스 크기가 붙은 뒤에야 됨) */
function drawClipPreviews() {
  Array.prototype.forEach.call(document.querySelectorAll('#lanes .sq-clip--midi'), drawMidiPreview);
}

/** 지금 펼쳐 둔 트랙 줄(.is-open)의 칸을 다시 채우기 — 줄 DOM 이 아니라 트랙 값으로 부릅니다 */
export function refreshOpenCells() {
  Array.prototype.forEach.call(document.querySelectorAll('#tracks > .sq-track.is-open'), function (row)
{
    const track = trackById(row.dataset.trackId);
    if (track) ensureLiveCells(track);
  });
}
```

문법 풀이

식	뜻
`clips = (list || []).slice()`	서버 배열을 그대로 복사해 모듈 안에 담습니다. 빈 값이면 빈 배열입니다
`String(c.track_id) === String(track.track_id)`	숫자와 글자가 섞여도 비교가 어긋나지 않게 둘 다 글자로 바꿉니다
`ticksToBars(c.start_tick)`	서버의 tick 을 편곡 화면이 쓰는 마디 수로 바꿉니다 (1박 = 960 tick)
`el.__notes = p.notes || []`	노트는 dataset 이 아니라 요소에 붙인 임시 값에 담습니다. 그리는 데만 쓰므로 화면에 노출되지 않습니다
`canvas.width = pw`	캔버스 안쪽 해상도를 기기 픽셀 배율(dpr)만큼 키워 선명하게 그립니다
`ctx.setTransform(dpr, 0, 0, dpr, 0, 0)`	이어서 그리는 좌표는 화면 크기 기준으로 두고, 배율은 한 번에 적용합니다

식	뜻
`row.dataset.trackId` 와 `trackById`	열려 있는 줄의 서버 번호를 읽어 트랙 값을 찾습니다
`Array.prototype.forEach.call(...)`	`querySelectorAll` 결과에 forEach 를 붙이는 전통 표기법입니다

4. 왜 이렇게

선택	이유
클립을 `dataset.clipId`, `dataset.patternId`, `dataset.sampleId` 로 담음	저장(PATCH · DELETE)과 패턴 노트 저장이 같은 요소에서 서버 번호를 읽습니다. 요소를 새로 만들 때마다 번호가 따라다닙니다
클립 이름은 `name`, 패턴 이름, `Sample #N` 순서로 결정	서버에 이름이 없어도 화면에 빈 칸이 보이지 않게 합니다
MIDI 노트를 DOM 이 아니라 canvas 에 그림	노트가 수백 개여도 요소 수가 늘지 않아 스크롤이 가볍습니다
`drawClipPreviews` 를 레이아웃 후에 한 번에 부름	클립을 만드는 시점에는 canvas 크기가 아직 0 입니다. 크기가 붙은 뒤에야 그릴 수 있습니다
`refreshOpenCells` 가 `.is-open` 인 줄만 다시 채움	이전에는 `forEach` 의 순번을 `ensureLiveCells` 에 넘기는 문제(0, 1, 2 가 트랙 번호로 읽힘)가 있어, 열려 있지 않은 칸까지 갱신되거나 트랙을 찾지 못했습니다. 줄의 `dataset.trackId` 로 트랙 값을 직접 찾도록 고쳤습니다
`indexPatterns` 은 `patternsById` 에, `indexClips` 은 배열에	패턴은 번호로 바로 찾고, 클립은 트랙별로 걸러내므로 배열이 편합니다

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 시험용 패턴을 만들고 화면에서 MIDI 클립을 놓은 뒤 새로 불러왔을 때 `data-clip-id` 와 `data-pattern-id` 가 남는지, MIDI 클립 안에 `canvas.sq-clip__preview` 에 노트 막대가 그려지는지, 연결을 저장한 뒤 트랙이 다시 그려져도 펼침 칸이 새 값으로 채워지는지 확인합니다. 시험은 14개 항목 모두 통과했고, 화면 사진은 `docs/screenshots/2026-10-19_s1-10a_clips_connections/` 에 있습니다.